

## **CS 501 E1 - Assignment 1: Mobility Lens**

### **1. Details:**

**Name:** Chris Mary Benson

**BUID:** U56085268

|                                  |                                |
|----------------------------------|--------------------------------|
| <b>Application Package Name:</b> | package com.chris.mobilitylens |
| <b>Emulator Model:</b>           | Pixel 6                        |
| <b>API Level:</b>                | 37.0                           |
| <b>minSdk:</b>                   | 24                             |
| <b>targetSdk:</b>                | 36                             |
| <b>compileSdk:</b>               | 37                             |

**Table 1: Android IDE Project Setup Details**

MinSdk: The minSdk correlated to the oldest Android API level that can install the run the application. Older versions < minSdk will not be able to run the application.

TargetSdk: The targetSdk specifies that the application is designed with the behaviours of specified Android API level kept in mind.

CompileSdk: The Android API level whose APIs and classes are available when compiling the code. This Android API level is used for the compilation of the application.

Files:

- MainActivity: app/src/main/java/com/chris/mobilitylens/[MainActivity.kt](#). This is the main entry point for the application.
- AndroidManifest.xml: app/manifests/AndroidManifest.xml. Contains information about the configurations and permissions required for the application.

- app-level Gradle file: app/build.gradle.kts. This file contains the dependencies required for the application such as the minSdk, targetSdk, etc.
- version catalog: gradlescripts/libs.versions.toml. This file manages the versions of the project's dependencies and plugins.
- Strings.xml: res/values/strings.xml. Stores reusable texts required by the application.

Mobile Design Decision: I used a vertical column layout for arranging the UI elements on the screen, which makes sense for a mobile since the screens are narrow and do not need to be spread across the screen as is done for a desktop application. This decision helped keep all the content organized within the screen and also supported vertical scrollability.

## 2. Output:

### (i) Using the 'Previous' and 'Next' buttons to toggle between different dimensions:

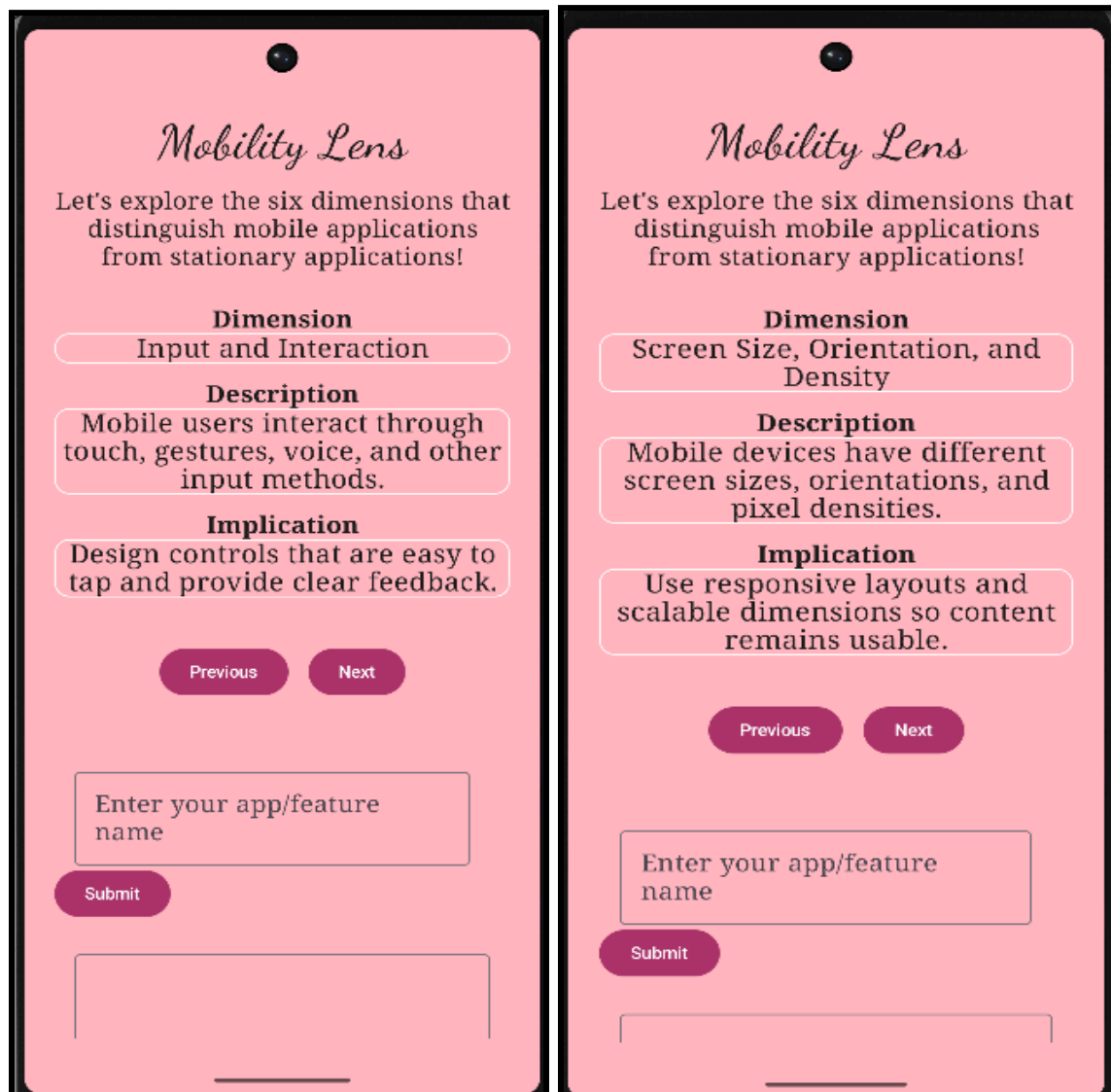
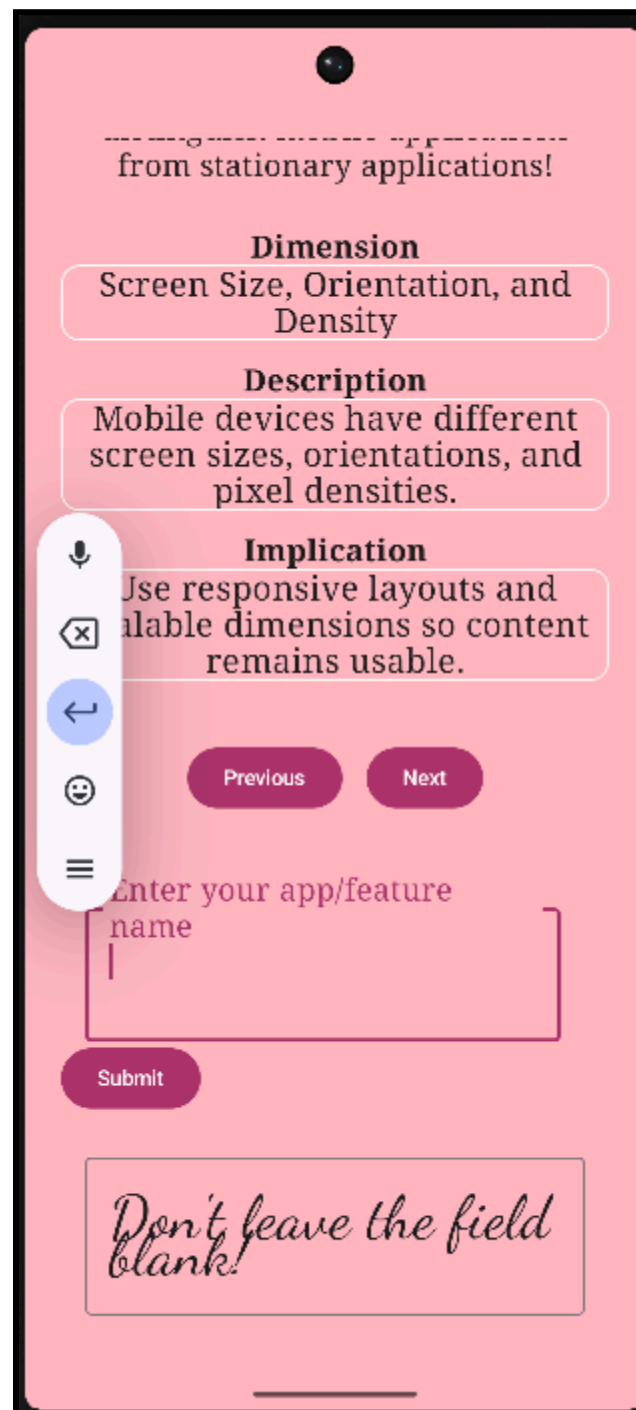


Figure 1: First Dimension

Figure 2: Second Dimension

**(ii) Feedback if 'Submit' button is clicked with empty input**



**Figure 3: Input Feedback**

**(iii) Output after user enters an app/feature name:**

**DIMENSION**

Lifecycle and Resource Constraints

**Description**

Mobile apps get interrupted or killed by the OS, and run on devices with limited battery, memory, and bandwidth.

**Implication**

Design for graceful state resumption and lean resource use.

Previous Next

Enter your app/feature name

Instagram

Submit

Instagram :  
Lifecycle and  
Resource  
Constraints

Figure 4: Entering a Valid Input

(iv) **Output clears when user clicks on the 'Previous' or 'Next' button:**

**Dimension**  
Security and Privacy Expectations

**Description**  
Mobile apps often access sensitive data like location, contacts, and camera. Request only what's needed and handle it transparently.

**Implication**  
Ask for permissions only when the feature is used, and explain why each one is needed.

Previous Next

Enter your app/feature name  
Instagram

Submit

Figure 5: Clearing Textbox when Navigating to Next Dimension

### 3. Rotation Observation

Before rotation:

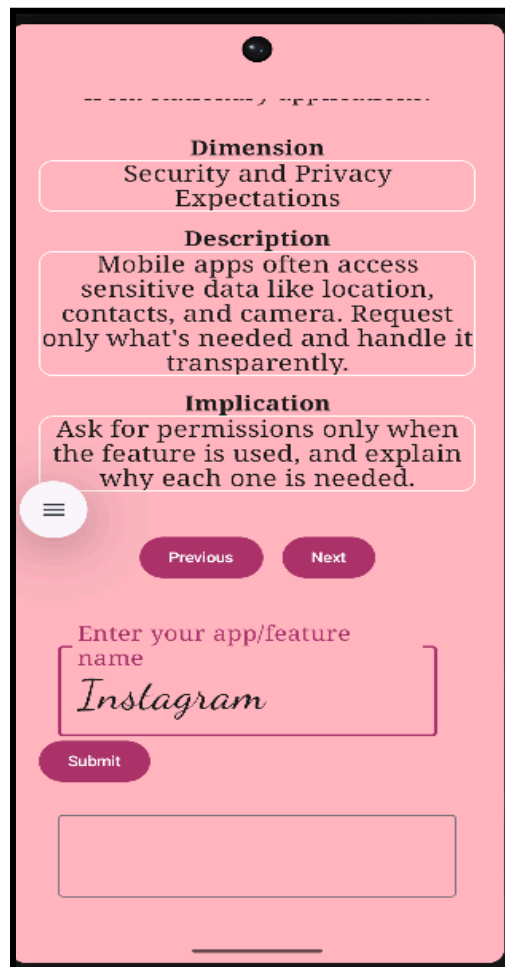
A vertical mobile app screen with a pink background. At the top, there is a dashed line and a small black circle. Below this, the text "Dimension" is followed by "Security and Privacy Expectations" in a rounded rectangle. Then, "Description" is followed by a paragraph: "Mobile apps often access sensitive data like location, contacts, and camera. Request only what's needed and handle it transparently." in a rounded rectangle. Next, "Implication" is followed by a paragraph: "Ask for permissions only when the feature is used, and explain why each one is needed." in a rounded rectangle. Below these is a white circle with three horizontal lines (a menu icon). Then, two rounded buttons labeled "Previous" and "Next" are shown. Below them is a text input field with the placeholder "Enter your app/feature name" and the text "Instagram" entered. Below the input field is a rounded button labeled "Submit". At the bottom, there is a large empty rounded rectangle. The screen is framed by a black border.

Figure 6: Before Rotating the Screen

After rotation:

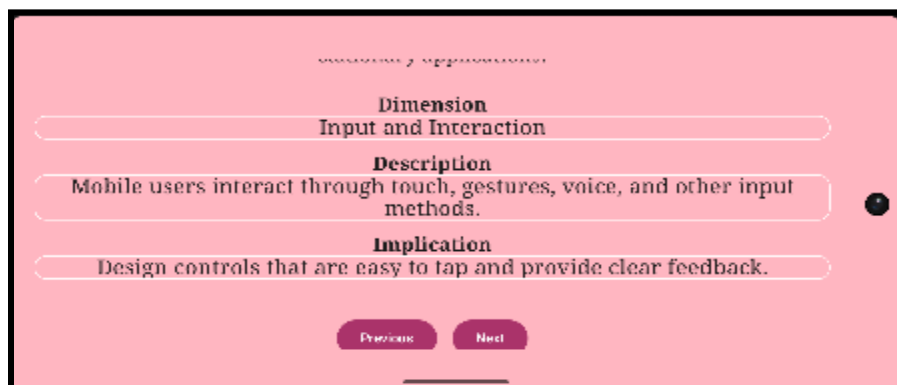
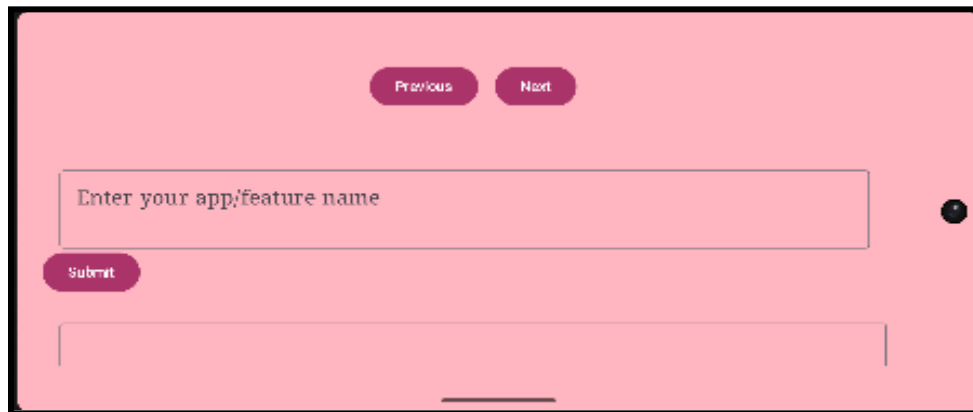
A horizontal mobile app screen with a pink background. At the top, there is a dashed line and a small black circle. Below this, the text "Dimension" is followed by "Input and Interaction" in a rounded rectangle. Then, "Description" is followed by a paragraph: "Mobile users interact through touch, gestures, voice, and other input methods." in a rounded rectangle. Next, "Implication" is followed by a paragraph: "Design controls that are easy to tap and provide clear feedback." in a rounded rectangle. Below these is a white circle with three horizontal lines (a menu icon). Then, two rounded buttons labeled "Previous" and "Next" are shown. At the bottom, there is a large empty rounded rectangle. The screen is framed by a black border.

Figure 6.1: After Rotating the Screen



**Figure 6.2: After Rotating the Screen**

We can see that after rotation, the screen has returned to its original state, displaying the first dimension. We would need to ensure that the state is preserved regardless of the orientation of the screen.

#### **4. Collaboration & AI Usage**

I used OpenAI ChatGPT and Claude during the development of the Mobility Lens application. I used AI for help in mainly using the components of Kotlin, as it is my first time using this programming language, as well as for help in how to navigate through the Android IDE.

My prompts to AI (Asked both AIs to make a summary of the prompts I used):

- Asked for explanations of the Android project structure and the purpose of files such as MainActivity, AndroidManifest.xml, Gradle files, the version catalog, and strings.xml.
- Asked for help understanding and implementing mutable state in Jetpack Compose.
- Asked how to implement the six mobility dimensions and Previous/Next navigation.
- Asked for explanations of UI formatting using Column, Row, Spacer, padding, and fillMaxWidth().
- Asked for explanations of minSdk, targetSdk, and compileSdk.
- Asked what a "customized Material theme" means.
- Asked what content: `@Composable () -> Unit` means.
- Asked me to explain each parameter in a colorScheme.

- Asked how to set a font for text.
- Asked where that font/typography config is defined in the project.
- Asked what alternatives exist to `FontFamily.Default`.
- Asked how to use two different fonts for two different `bodyLarge`-style texts.
- Asked how to center text.
- Pasted a Kotlin code snippet (title/intro Text composables) for review.
- Reported a "stringResource unresolved" error and asked for help.

Relevant suggestions produced: ChatGPT suggested using `remember` and `mutableStateOf` for Compose state, `Column` and `Row` for arranging UI elements, `Spacer` and padding for formatting, and `Button` and `OutlinedTextField` for user interaction.